DOCE-102 Statistical Computing
Unit 3: Pseudo-randomness and IEEE754

Nicolás Rivera

1S 2022

Universidad de Valparaíso
CHILE

Stochastic Simulation

The goal of Stochastic Simulation is to simulate random variables and stochastic processes based on a sequence $(U_i)_{i\geq 1}$ of uniform random variables in $[0, 1]$

Recall that for $0 \leq a < b \leq 1$ we have that

$$\mathbb{P}(a < U_i < b) = b - a$$

In Python we can use `np.random.uniform(0,1,n` to generate a vector of n independent uniform random variables.

For theoretical and practical purposes we can assume that we have at our disposal a sequence of independent $\mathcal{U}(0,1)$ random variables

Nevertheless, in some very rare and specific cases, we may actually have some practical problems due to the following incompatible claims (which are both true)

1. For $U \sim \mathcal{U}(0,1)$,then $\mathbb{P}(U \text{ is irrational}) = 1$
2. A computer can only store a finite number of decimals

Pseudo-Randomness

Pseudo-Randomness

(Informal) Definition

A sequence of pseudo-random numbers $(U_i)_{i=1}^{\infty}$ is a deterministic sequence of numbers in $[0, 1]$, having the same relevant statistical properties as a sequence of independent uniform random numbers.
A sequence of pseudo-random bits $(B_i)_{i=1}^{\infty}$ is a sequence of numbers taking values 0 or 1 having the same relevant statistical properties as a sequence of random bits.

1. Defining what is random or pseudo-random is very hard in theory and in practice.
2. From the theoretical point of view, there are notions such as **Kolmogorov Complexity**
3. In practice, we are happy if the numbers looks like we expect from our probability theory class.
4. The main issue is to obtain a sequence U_i such that there are no clear patterns
5. A lot of research on this in the past. Right now is essentially a solved problem
6. `np.random.uniform(0,1,n)` generates a vector of n random numbers in Python. Most programming languages have their own system.
7. To ensure **reproduciblity** use a seed. In Python we can use `np.random.seed()` set the seed m.

Pseudo-Randomness

8. Python automatically sets a seed, but it changes when we start Python again.
9. Once a seed is set a long-sequence of pseudo-random bits is constructed
10. In a 64 bits machine, a random uniform in $[0, 1]$ uses exactly 52 pseudo-random bits. We will see why

Detour: A bit of double-precision floating-point

A bit of Double-precision floating-point

In a computer number are stored as bits. Number are usually stored in what is called a **double** float number.

In Python numbers are an object *numeric*, but there are different types of numbers such as integer, float, etc.

```
print(type(4))
print(type(4.0))
```

For many users this makes no much difference, but it does at low levels

A bit of Double-precision floating-point

WARNING.

If we are actually working with integers, it is convenient to tell Python (and in general any language) that. In Python we can write

```
a = type(int(4.0)) #transform float into int
print(type(a))
print(a)
```

A bit of Double-precision floating-point

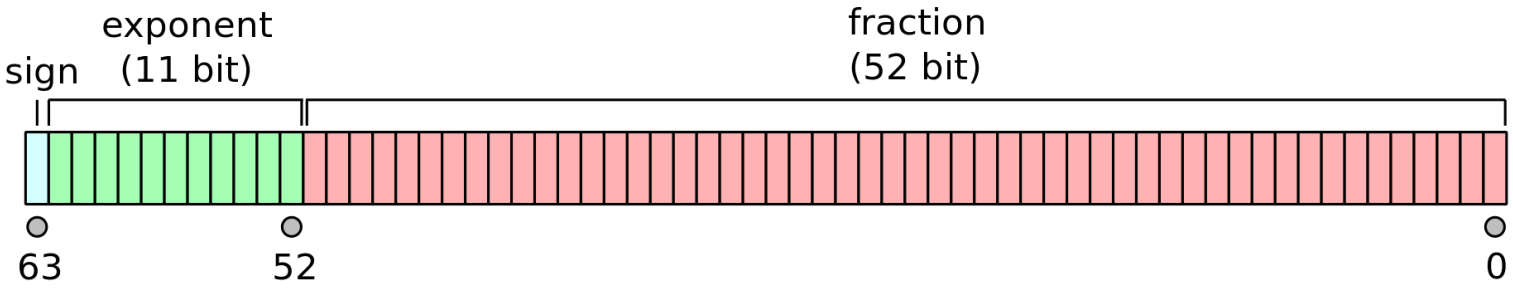
So, what is the difference?

1. *integer:* can only manipulate integers. The binary representation is very natural, and several algorithms are specialised for these type of data structure. Also, integer in Python have unlimited size (the only limit is the memory available in the computer)
2. *double:* can manipulate rational/*real* numbers. The representation in Python is the **IEEE 754 standard**, also known as **binary64**. It uses 64 bits = 8 bytes. Most Languages work with this standard

A bit of Double-precision floating-point

In the IEEE 754 standard we allow 64 bits to represent a number.

- The first bit (from left to right) represent the ***sign*** of the number
- The next 11 bits represent the ***exponent***
- The next 52 represent the ***fraction***



A bit of Double-precision floating-point

- **sign (S):** 0 if number is positive, 1 if negative.
- **exponent (E):** is a positive integer between 0 an $2^{11} - 1 = 2047$
- **fraction (F):** Let $b[0], \ldots b[51]$ represent the remaining 52 bits (from left to right), then the fraction represents the number (in binary)

$$(1.b[1]b[2] \cdots b[51]b[52])_2$$

  or equivalently (in decimal)

$$F = 1 + \sum_{i=1}^{52} b[i]2^{-i}$$

Then, the number represented by a double is

$$(-1)^{\text{sign}} \times F \times 2^{E-1023}$$

Simple Summary

We write a number F in binary of the form $1.xxxxxxx$, and then we move the *comma* ($\approx$ 1000 spaces to the right or to the left)

A bit of Double-precision floating-point

Some commments:

1. In decimal expansion, the fraction has approximately 15 to 17 digits of precision, but sometimes more
2. The exponent allow us to move the *comma* around 308 positions to the left or right
3. We need special encoding to number such as 0

```
#a hack to get the binary representation of a float
def float_to_bits(x):
    bits = np.frombuffer(np.array(x, dtype=np.float64).tobytes(), dtype=np.uint64)[0]
    binary_str = format(bits, '064b')
    return(binary_str)

print(float_to_bits(4.0))

#Special encoding
print(float_to_bits(0))
print(float_to_bits('inf'))
print(float_to_bits('-inf'))
print(float_to_bits('nan'))
```

A bit of Double-precision floating-point

A word of caution:

1. As we can see, everything is an approximation.
2. This lead to some well-known problems

```
print((0.1+0.2)==0.3)
print(0.1+0.2)
```

   There is certainly something wrong. .Let's try

```
print(float_to_bits(0.1+0.2))
print(float_to_bits(0.3))
print(0.3-(0.1+0.2))
```

3. Be careful with `R`. It always round number when writing them on the screen.
4. In practice, when comparing double just do

```
np.abs(0.1+0.2-0.3)<0.00000001
```

Take home message

While there may be numerical errors in our computation, they tend to appear in very far away in our decimal expansion. **We should not worry about this**, but we have to **know** that in some specific cases and applications we may have issues, and we have to be **aware** of that.